ADVANCED ROBOTIC PERCEPTION / IMAGE PROCESSING AND COMPUTER VISION

SEQUENCES AND TRANSFORMERS

RIKKE GADE 2026

SLIDES BY MOHAMMAD SABET

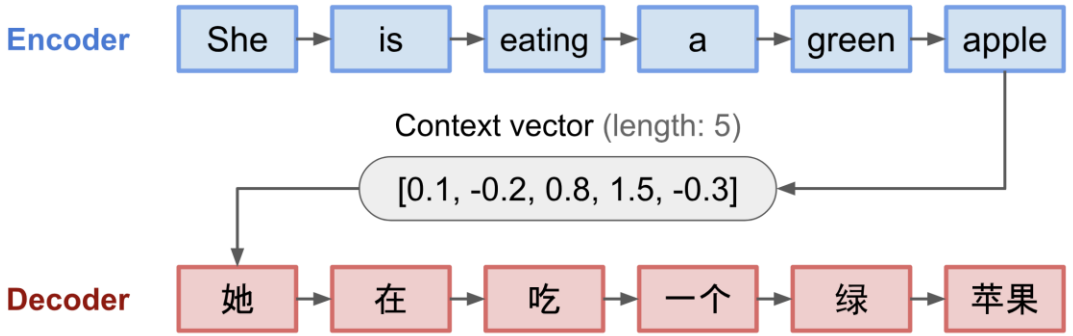


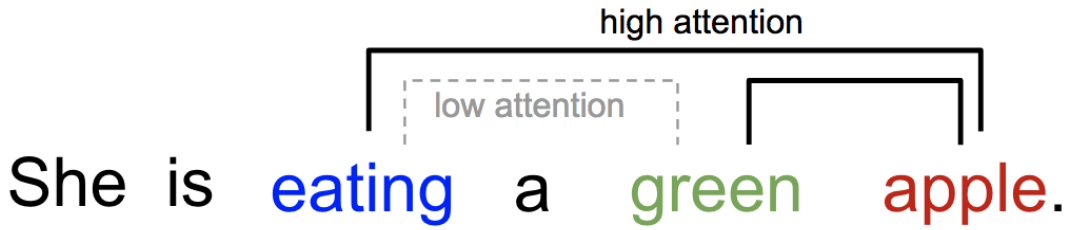
AALBORG UNIVERSITY
DENMARK

Agenda

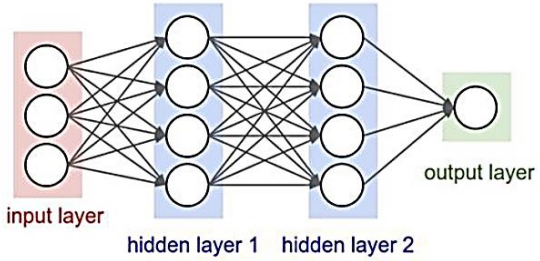
Sequence Modeling with RNNs
- Limitations of CNNs for sequential data
- Recurrent Neural Networks fundamentals
- RNN architecture and information flow
- LSTM cells for long-term memory

Transformers & Vision
- Transformer architecture overview
- *Details provided in videos: 3Blue1Brown!*

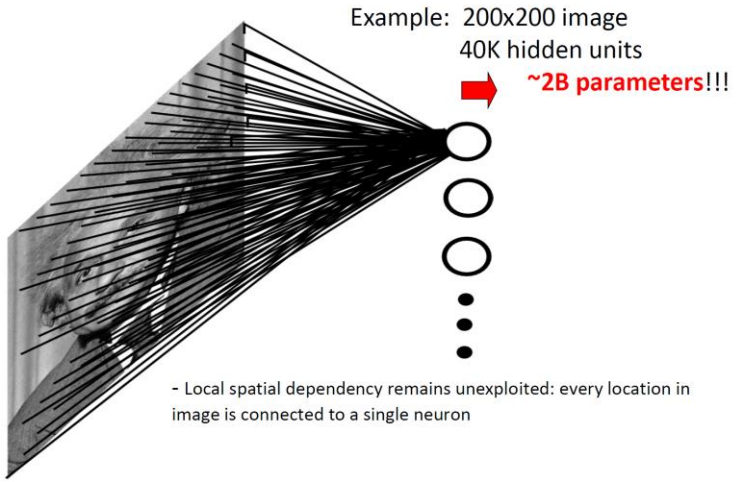


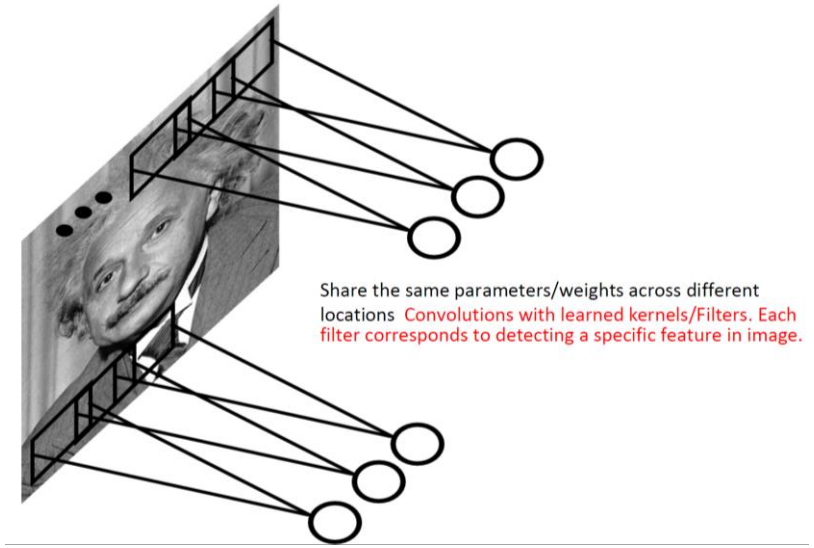


Recap : Neural network Design: FCN vs CNN




```
# forward-pass of a 3-layer neural network:
f = lambda x: 1.0/(1.0 + np.exp(-x)) # activation function (use sigmoid)
x = np.random.randn(3, 1) # random input vector of three numbers (3x1)
h1 = f(np.dot(W1, x) + b1) # calculate first hidden layer activations (4x1)
h2 = f(np.dot(W2, h1) + b2) # calculate second hidden layer activations (4x1)
out = np.dot(W3, h2) + b3 # output neuron (1x1)
```





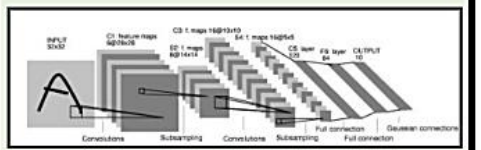
Neural Network (NN) : Feed all features at once.

Fully Connected Neural Network : FCN

Convolutional Neural Network : CNN

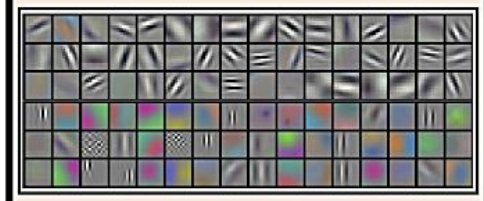
Recap: Neural Network Design Play

Convolutions



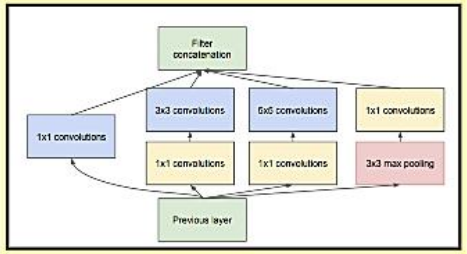
LeCun et al.,
LeNet, 1998

ILSVRC12 Scale up (with GPUs)



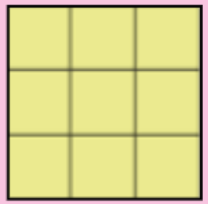
Krizhevsky et al.,
AlexNet, 2012

ILSVRC14 Multiscale



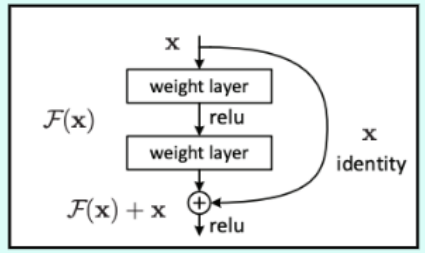
Szegedy et al.,
Inception, 2015

ILSVRC14 3x3 convs



Simonyan and Zisserman,
VGGNets, 2015

ILSVRC15 Residuals

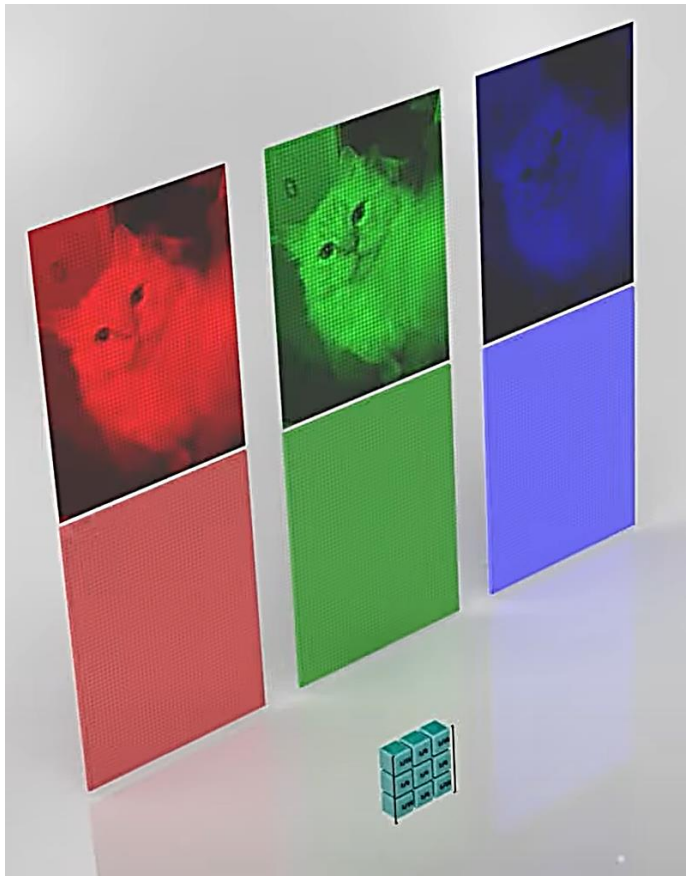


He et al.,
ResNet, 2016

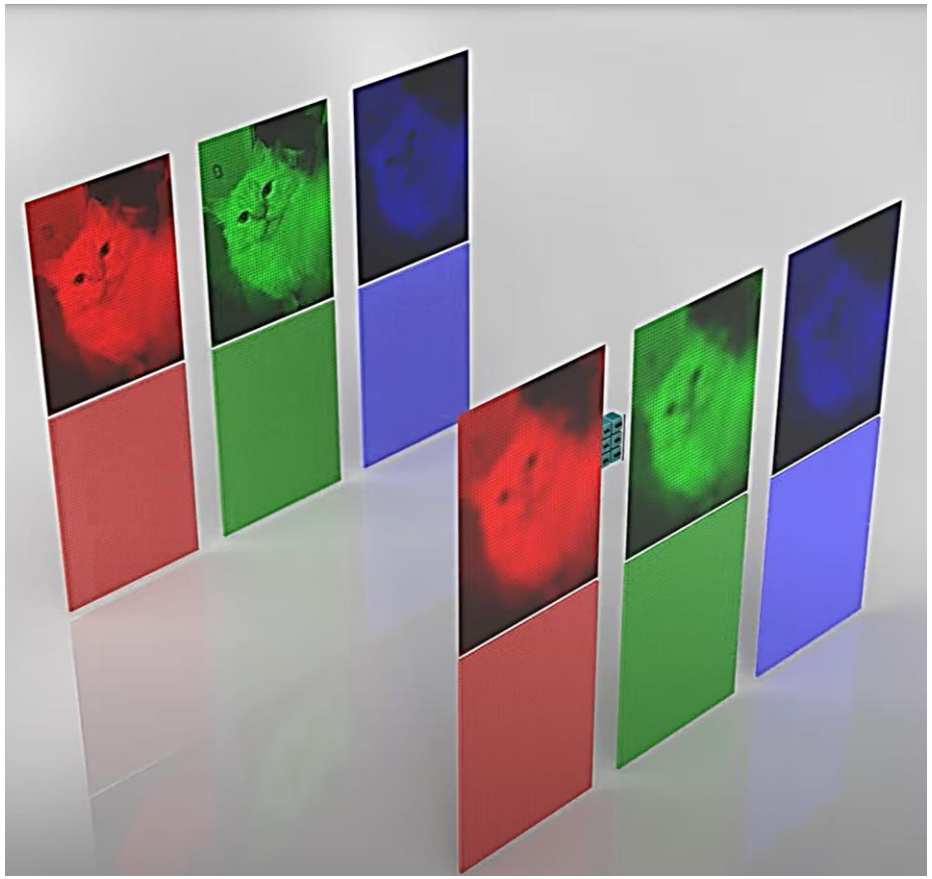
- The ImageNet Large Scale Visual Recognition Challenge (ILSVRC) evaluates algorithms for object detection and image classification at large scale.

Convolution : Image processing vs Neural Network

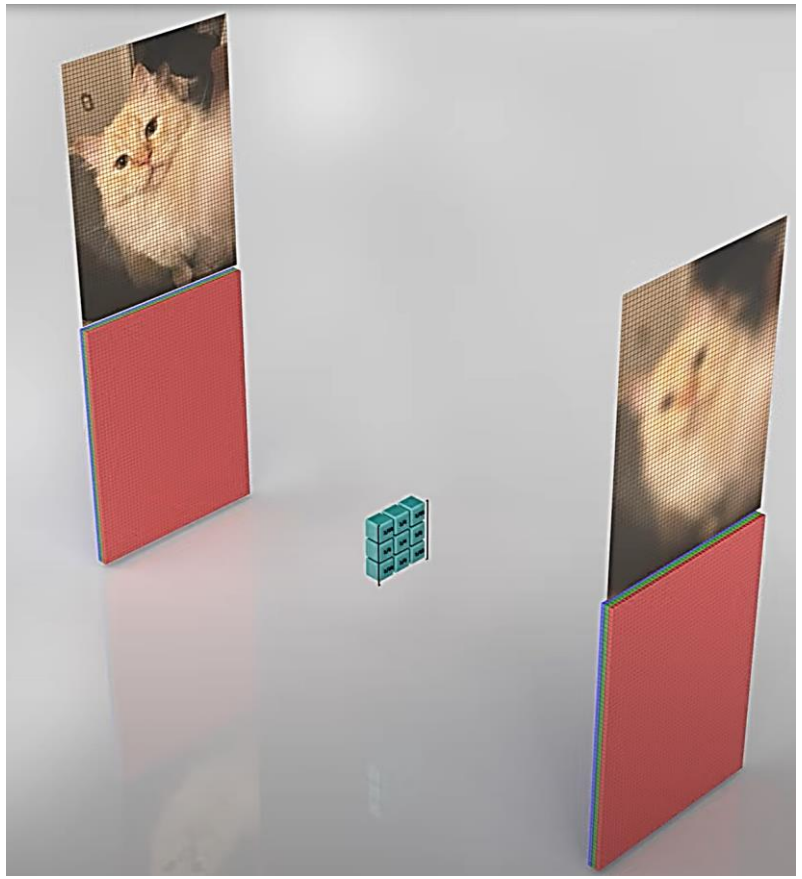
- In image processing an image is separated to **three** 2D input (RGB) and 2D CONV filter is applied to each channel. Then each computed channel are stack together to reshape to same 3D image size.



- Separate the channel: 3D to three 2D



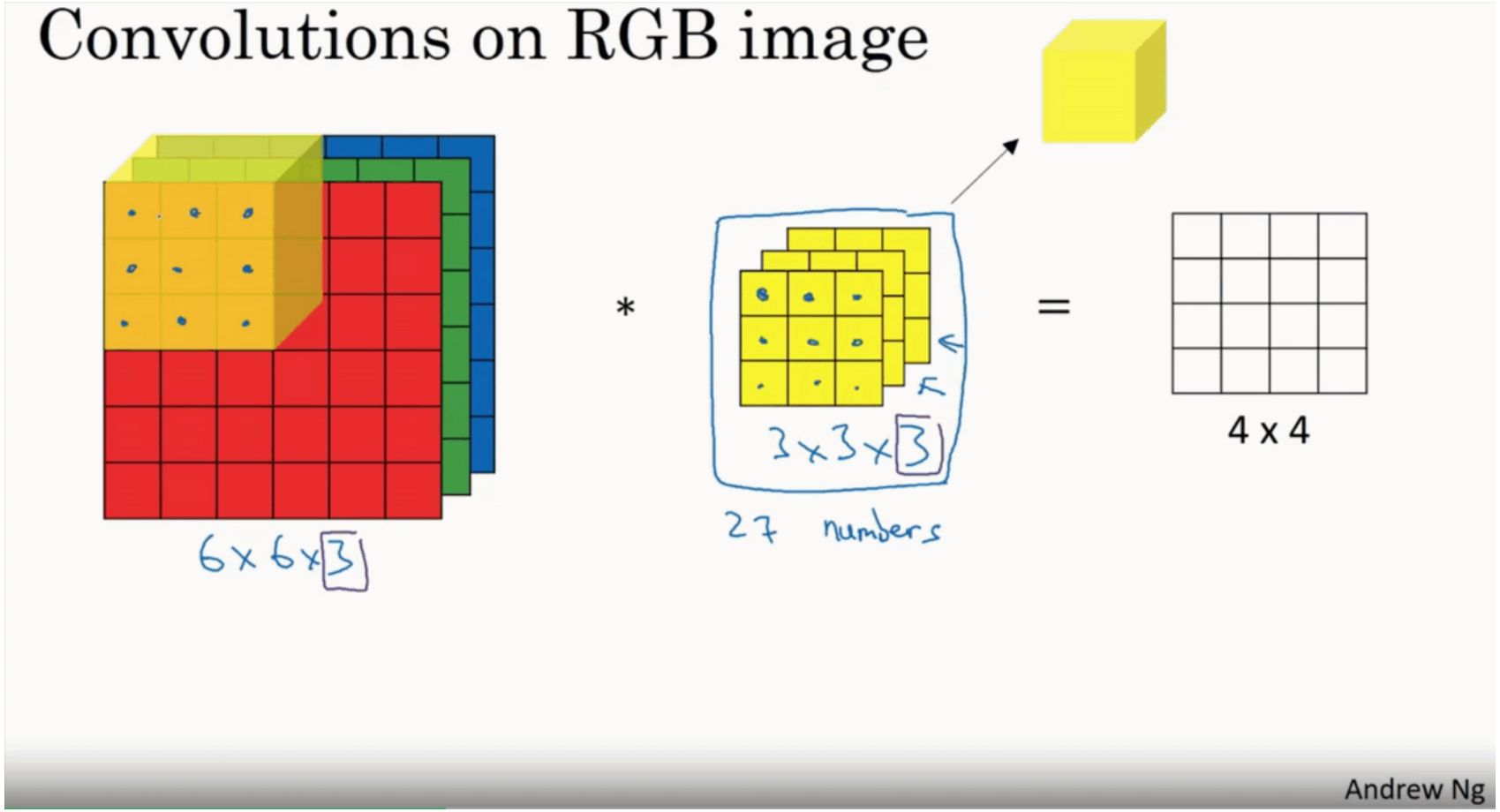
- Perform Conv on each channel (RGB)



- Stack the three 2D convolved result to 3D image.

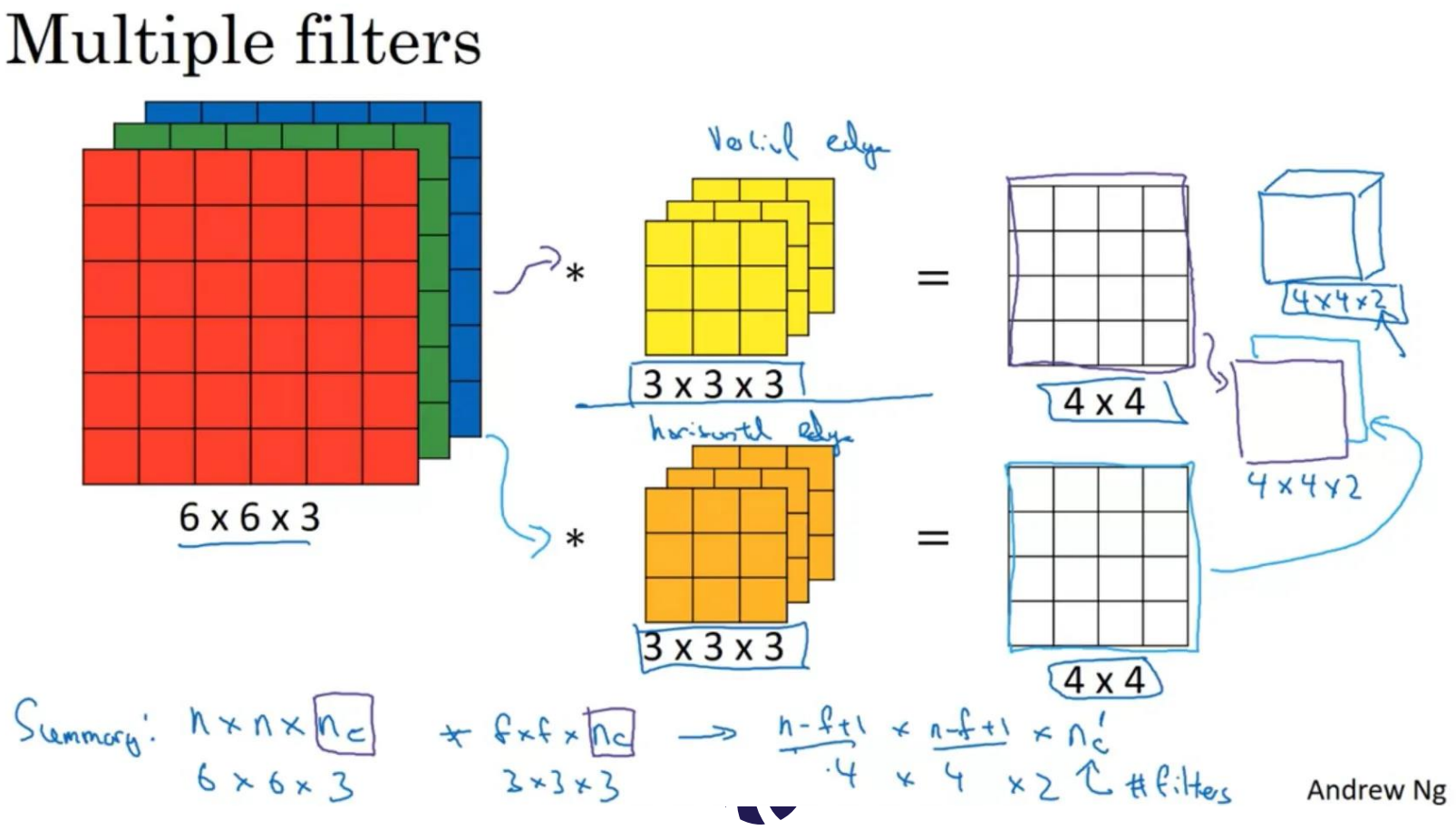
Convolution : Neural Network

- In Neural Network the size of filter should match the depth of input image. The depth of an image corresponds to different channel in image (RGB, Depth =3). For an input with M-Dimensional depth, the filter Depth should be M!



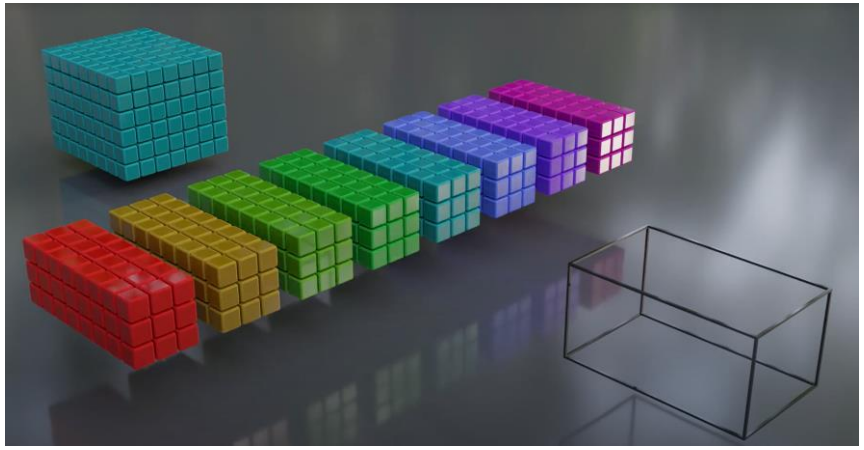
Convolution : Neural Network

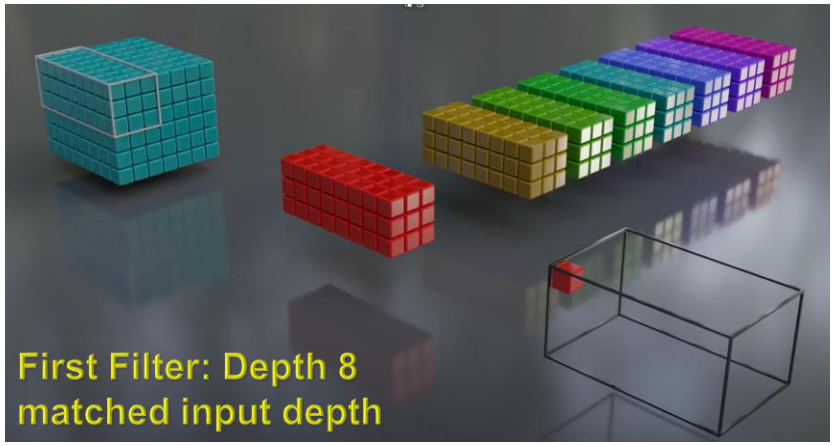
- In Neural Network the size of filter should match the depth of input image. The depth of an image corresponds to different channel in image (RGB, Depth =3). For an input with M-Dimensional depth, the filter Depth should be M!

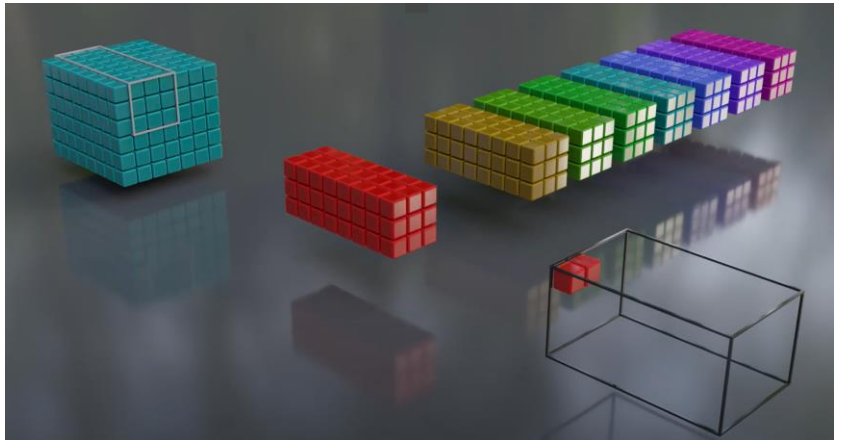


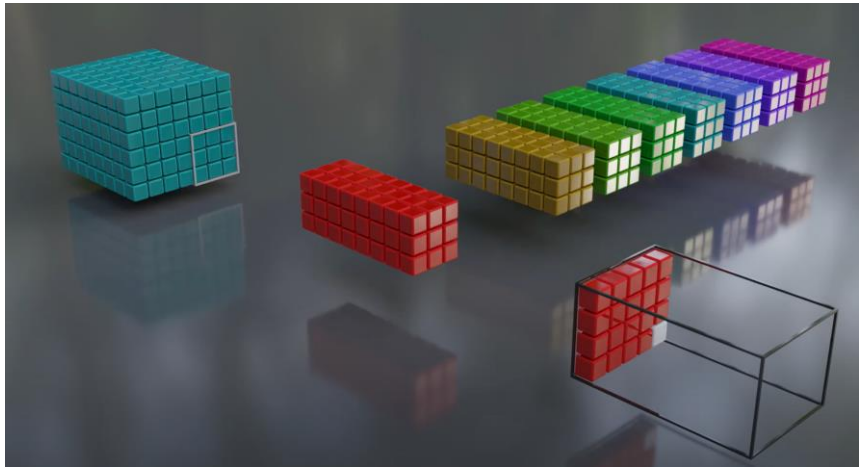
Convolution : Neural Network

- In Neural Network the size of filter should match the depth of input image. The depth of an image corresponds to different channel in image (RGB, Depth =3). For an input with M-Dimensional depth, the filter Depth should be M!



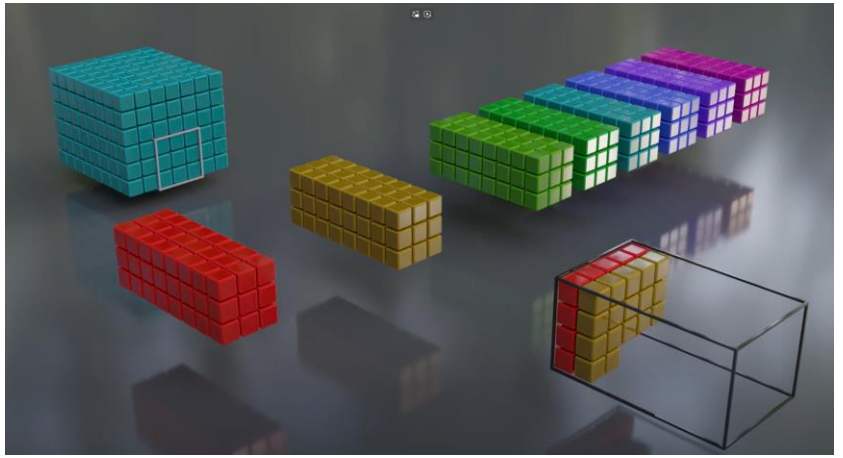






Convolution is completed for first filter continue the same operation for next filter (in yellow).

$$a = \sum_{i=1}^{n} x_i w_i + b$$



Today's Lecture: Sequence Challenge

- So far we assumed that all inputs applied to any Deep Neural Network (DNN) are stationary and fixed-length (e.g., Image W*H*3).
- What if we have a sequential and **time index** data like text or video frames?
- We need to have some neural network architectures that can incorporate time or in general be able to do sequence modeling
- Simple models commonly used before the era of transformers.
  - RNNs and some Variants

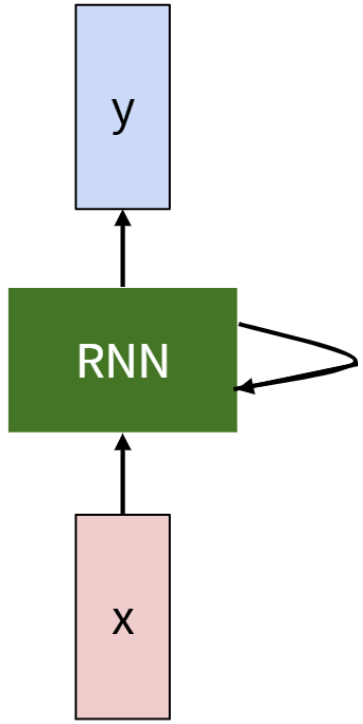


Recurrent Neural Network (RNN)

- **RNNs process input step by step**, feeding in one feature (or token) at a time across time steps.

- At each time step (except the first), the model takes two inputs:

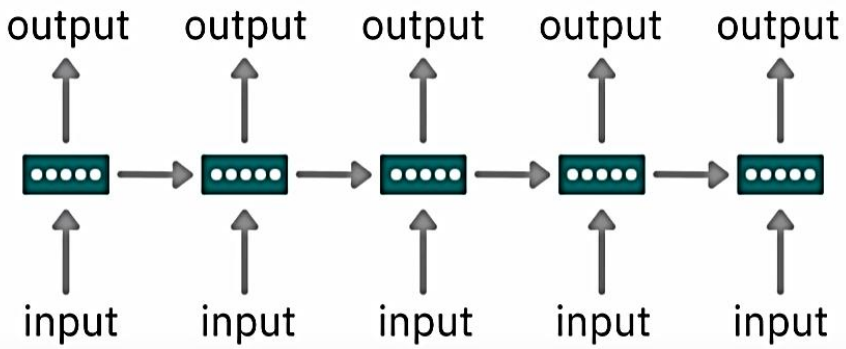
  •the current input at that time step
  •the output/hidden state from the previous time step



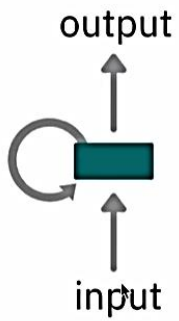
Recurrent Neural Network (RNN)

- An RNN is actually a **single cell that processes a sequence one step at a time**, feeding its output back into itself. But to visualize and compute gradients, we "unroll" it across time:

- **Rolled (actual)**: One cell → processes x_0, then x_1, then x_2, etc. sequentially
- **Unrolled (visualization)**: Shows that same cell as if it were 5 separate cells side-by-side
- Think of it like unrolling a loop in programming to see each iteration laid out.



• Unrolled RNN



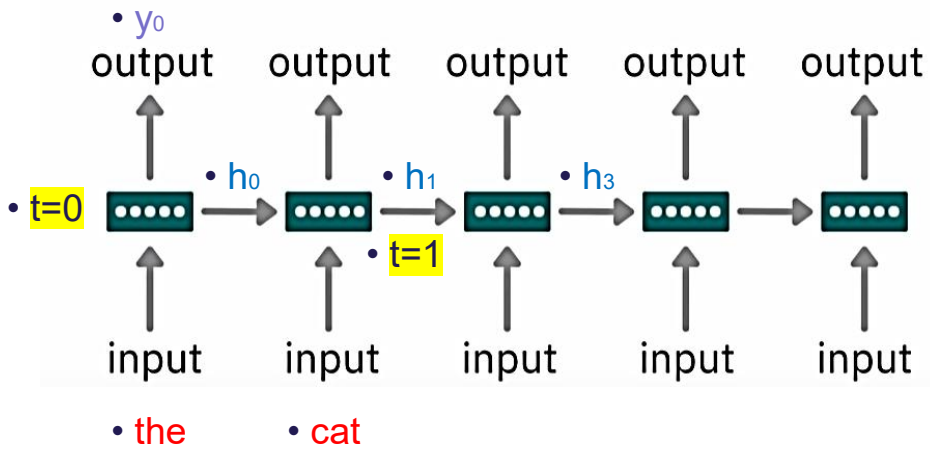
The same weights are used at every timestep

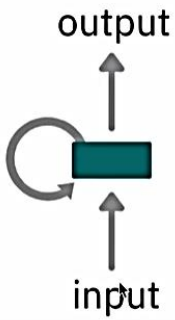
• Single cell RNN: it has "internal state" that is updated as a sequence is processed.

Recurrent Neural Network (RNN)

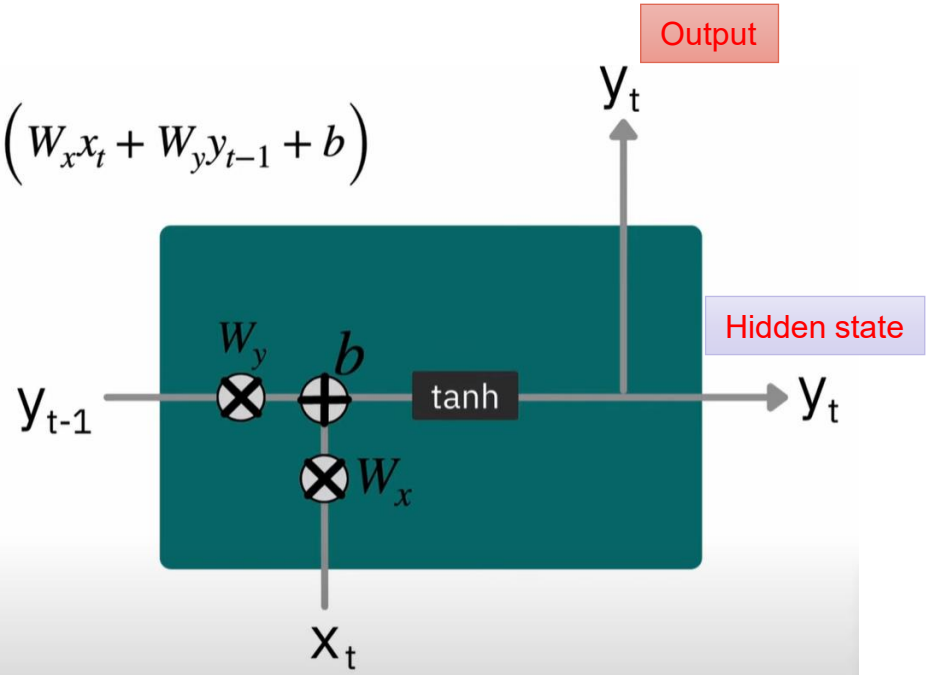
Example: 4 words "The cat sat down"

t=0: Input "The" → RNN cell → hidden state h_0, output y_0
↓ (h_0 passed forward)
t=1: Input "cat" → RNN cell → hidden state h_1, output y_1
↓ (h_1 passed forward)
t=2: Input "sat" → RNN cell → hidden state h_2, output y_2
↓ (h_2 passed forward)
t=3: Input "down" → RNN cell → hidden state h_3, output y_3





$$y_t = \alpha\left(W_x x_t + W_y y_{t-1} + b\right)$$

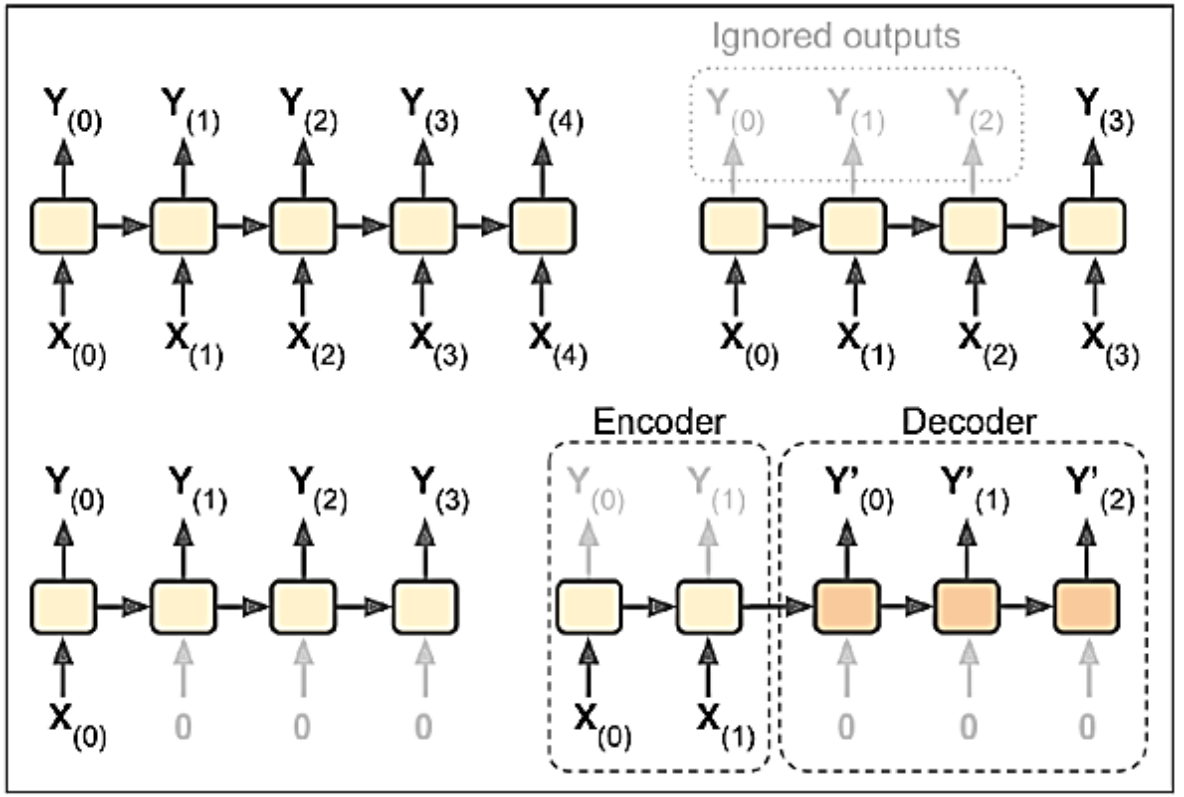


AALBORG UNIVERSITY
DENMARK

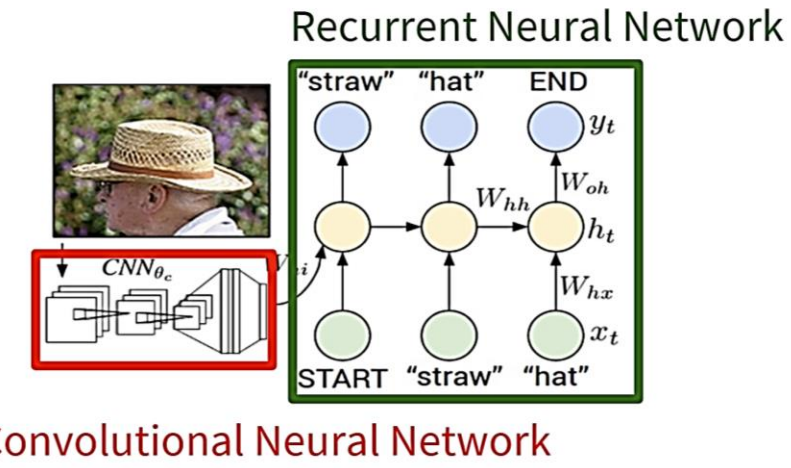
RNN Types: Input and Output Sequences

Examples:

- Seq-to-seq (top left): Stock price forcasting
- Seq-to-vector (top right) : Sequence classification
- Vector-to-seq (bottom left) : Image captioning
- Encoder–Decoder (bottom right) :Translation



RNN application: Image Captioning





A cat sitting on a suitcase on the floor



A cat is sitting on a tree branch



A dog is running in the grass with a frisbee



A white teddy bear sitting in the grass



Two people walking on the beach with surfboards



A tennis player in action on the court



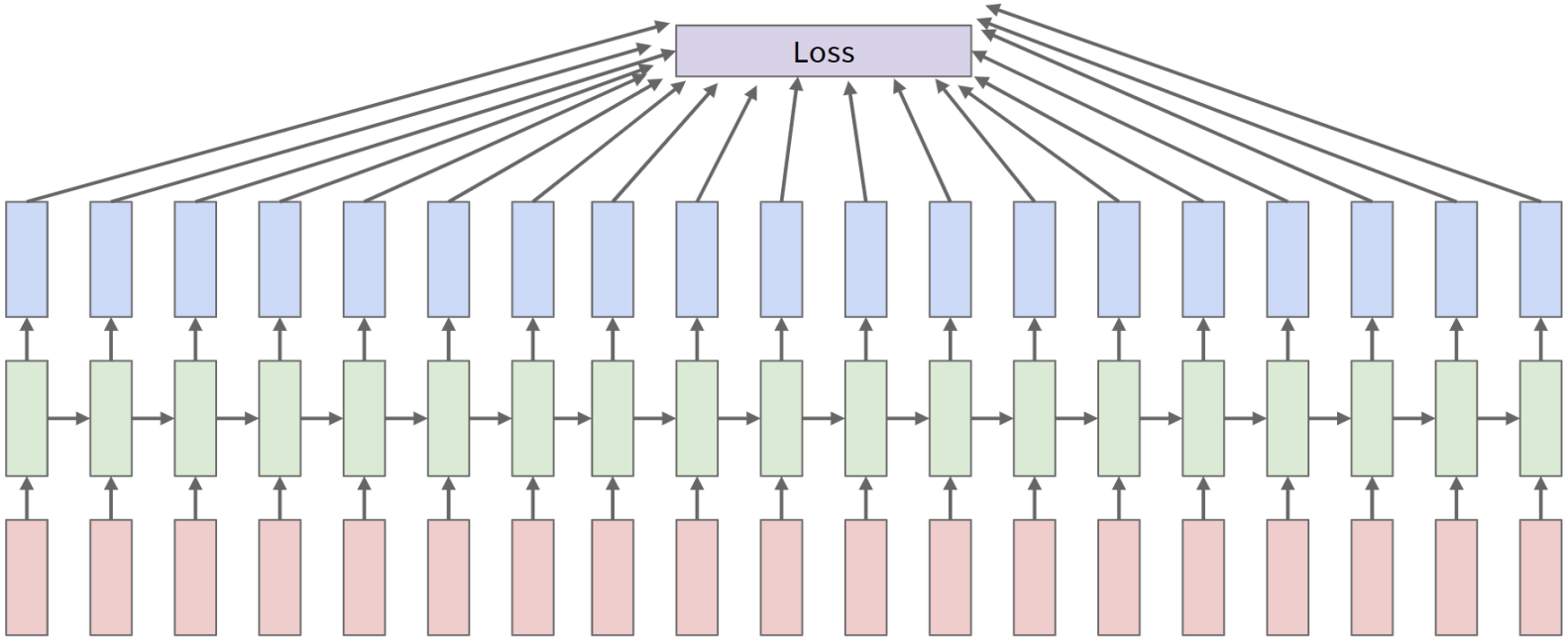
Two giraffes standing in a grassy field



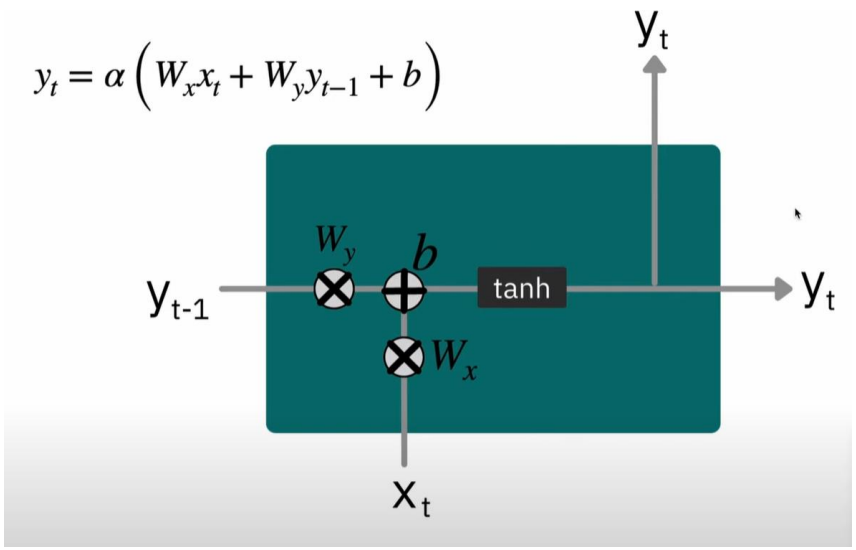
A man riding a dirt bike on a dirt track

RNN: Limitations

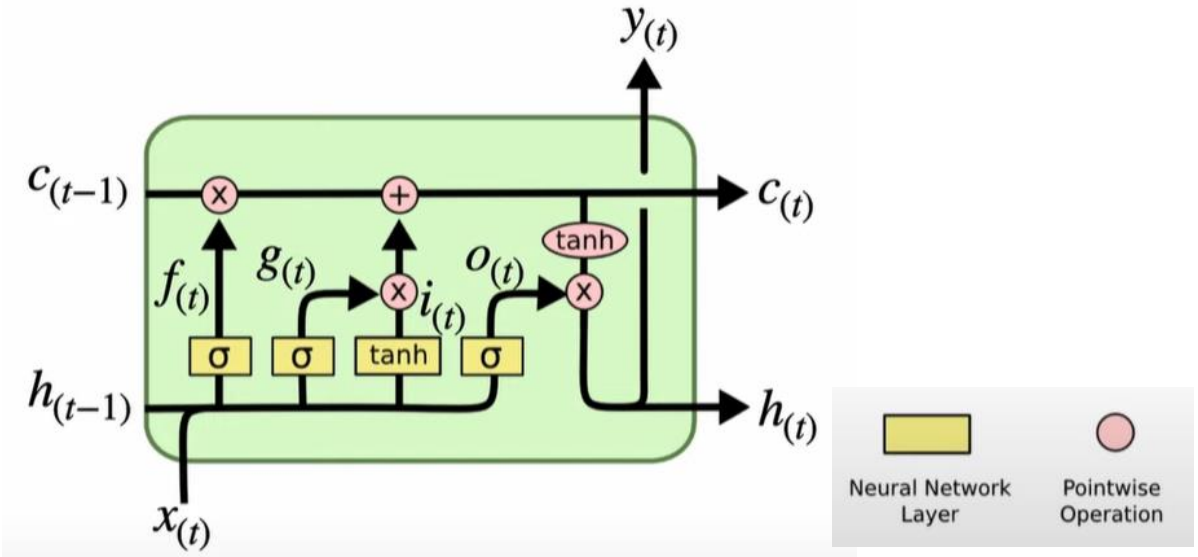
- Unstable gradient, vanishing gradient
- Recurrent computation is slow
- Another problem is it does not work for long context or long-term memory: It keeps forgetting
- For this we use LSTM or GRU cell to make sure we still remember the beginning of our sequence



Long Short-Term Memory (LSTM)



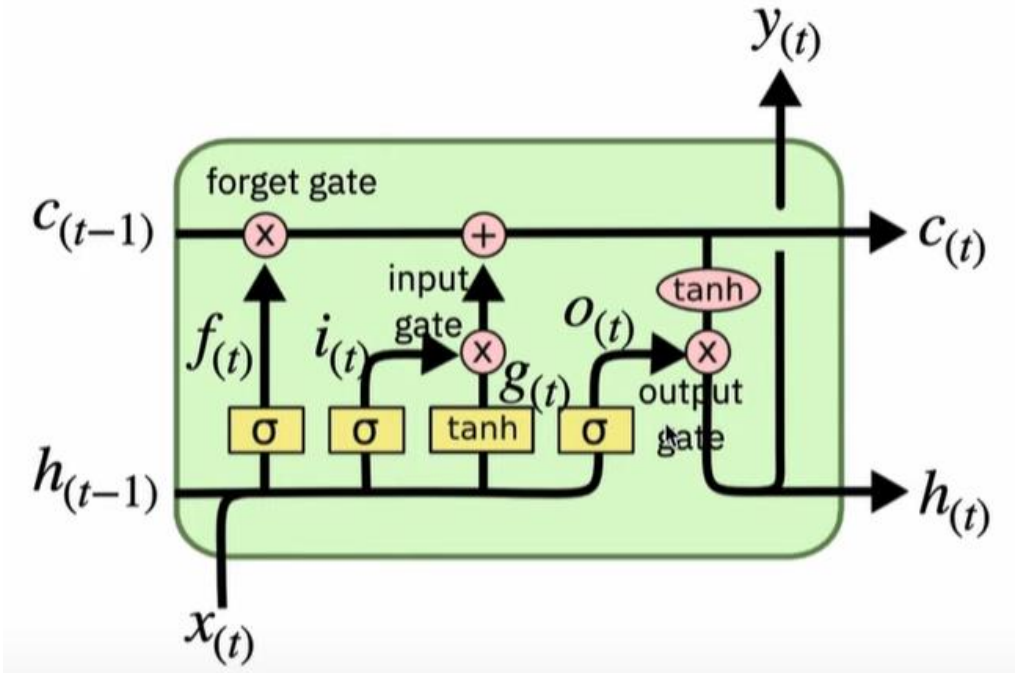
RNN Cell



LSTM Cell

Long Short-Term Memory (LSTM)

- The **first input** goes through calculations with activations.
- ***c (cell state)*** acts as the long-term memory. Along this path, fewer changes happen compared to the hidden state path.
- The ***hidden state (h)*** is updated more actively — information can be *forgotten* or *added* before being passed to the next time step



How do we know what to forget, add, or output?

Controlled by the **gates** (sigmoid values between 0 and 1):

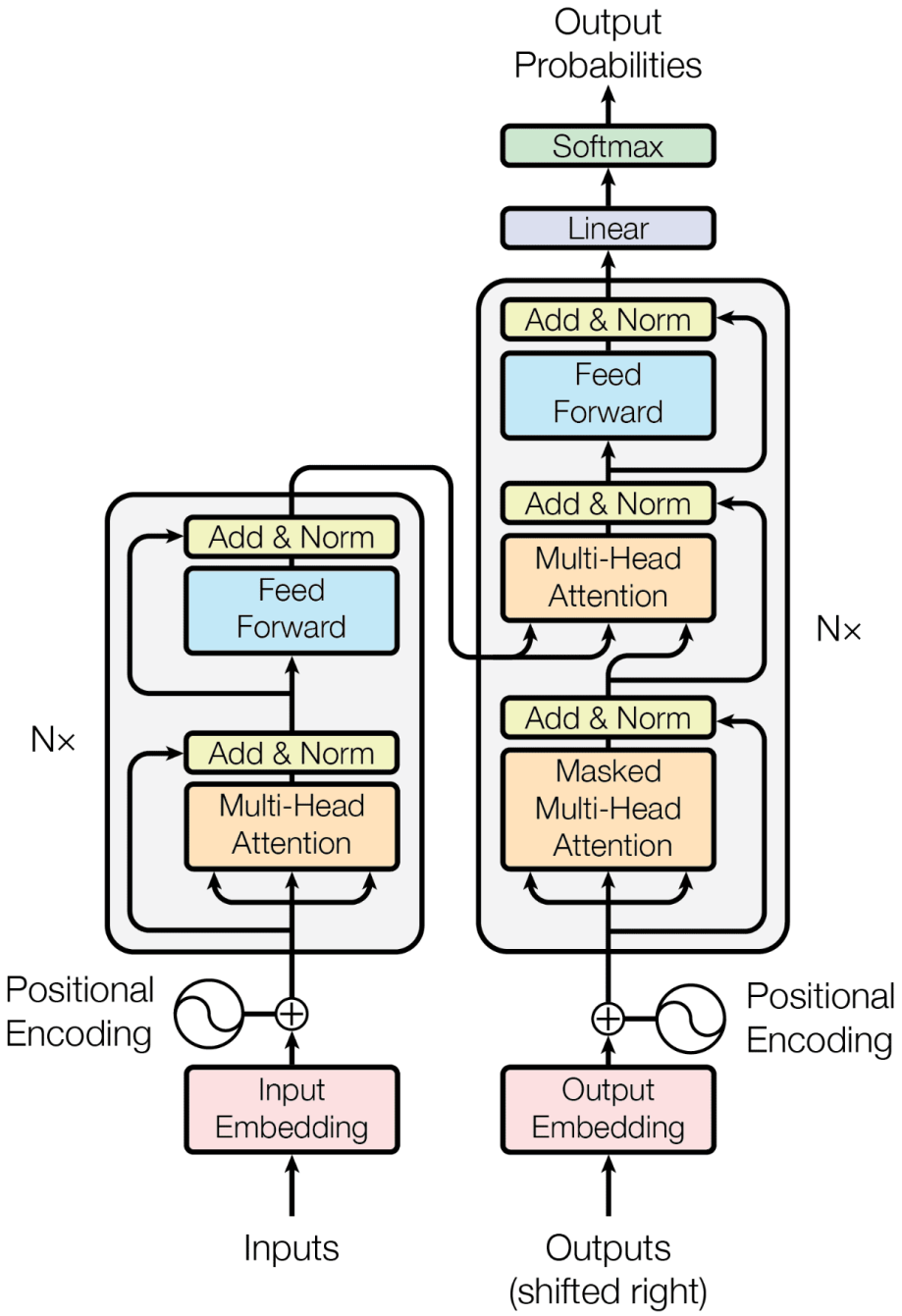
- **Forget gate** → decides which parts of the long-term memory to remove.
- **Input gate** → decides which new information
- **Output gate** → decides which parts of the memory should influence the hidden state/output at each time step.

Summary

- Vanilla RNNs are simple but often ineffective
- Backward gradients in RNNs can vanish or explode

- Improved variants like LSTMs are still widely used:
  - Improved long-term dependency
  - Mitigates vanishing gradients
  - Flexible information flow

- RNNs process input one step at a time, which is slow and not well-suited for longer contexts → this inspired the development of **Transformers,** designed for parallelism and long-range dependencies





Transformers!

arXiv:1706.03762v1 [cs.CL] 12 Jun 2017

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com
Noam Shazeer* Google Brain noam@google.com
Niki Parmar* Google Research nikip@google.com
Jakob Uszkoreit* Google Research usz@google.com
Llion Jones* Google Research llion@google.com
Aidan N. Gomez*† University of Toronto aidan@cs.toronto.edu
Łukasz Kaiser* Google Brain lukaszkaiser@google.com
Illia Polosukhin* illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks in an encoder-decoder configuration. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.0 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

1 Introduction

Recurrent neural networks, long short-term memory [12] and gated recurrent [7] neural networks in particular, have been firmly established as state of the art approaches in sequence modeling and transduction problems such as language modeling and machine translation [31, 2, 5]. Numerous efforts have since continued to push the boundaries of recurrent language models and encoder-decoder architectures [34, 22, 14].

Natural Language Processing

Machine Translation

Why did it take 3 years?

Why Google?

2010.11929v1 [cs.CV] 22 Oct 2020

An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale

Alexey Dosovitskiy*,†, Lucas Beyer*, Alexander Kolesnikov*, Dirk Weissenborn*, Xiaohua Zhai*, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby*,†
*equal technical contribution, †equal advising
Google Research, Brain Team
{adosovitskiy, neilhoulsby}@google.com

Abstract

While the Transformer architecture has become the de-facto standard for natural language processing tasks, its applications to computer vision remain limited. In vision, attention is either applied in conjunction with convolutional networks, or used to replace certain components of convolutional networks while keeping their overall structure in place. We show that this reliance on CNNs is not necessary and a pure transformer applied directly to sequences of image patches can perform very well on image classification tasks. When pre-trained on large amounts of data and transferred to multiple mid-sized or small image recognition benchmarks (ImageNet, CIFAR-100, VTAB, etc.), Vision Transformer (ViT) attains excellent results compared to state-of-the-art convolutional networks while requiring substantially fewer computational resources to train.[1]

1 Introduction

Self-attention-based architectures, in particular Transformers (Vaswani et al., 2017), have become the model of choice in natural language processing (NLP). The dominant approach is to pre-train on a large text corpus and then fine-tune on a smaller task-specific dataset (Devlin et al., 2019). Thanks to Transformers' computational efficiency and scalability, it has become possible to train models of unprecedented size, with over 100B parameters. With the models and datasets growing, there is still no sign of saturating performance.

Computer Vision

Vision Transformers (ViTs)

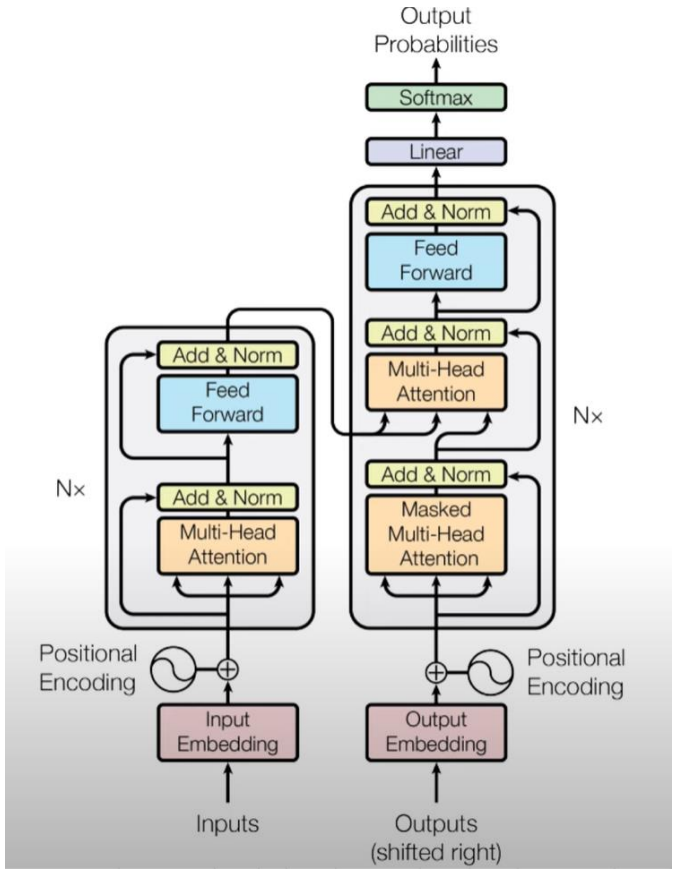
What is a transformer?

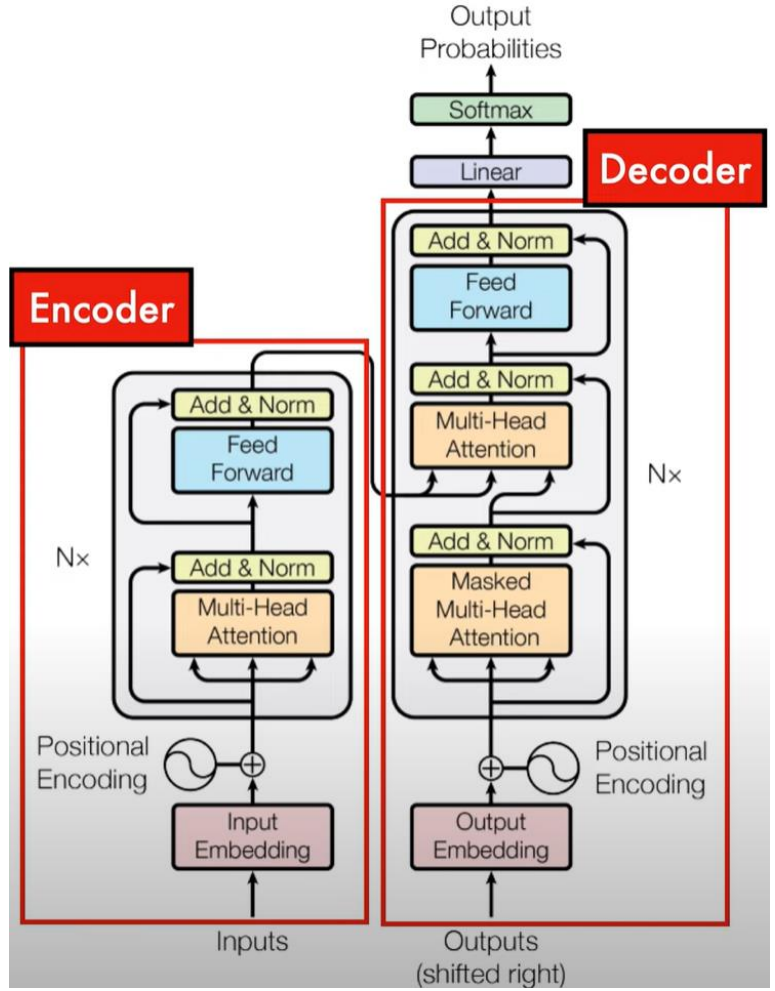
Encoder: learns useful representation of input

Decoder: "decodes" encoded representation and combines with other input to predict output

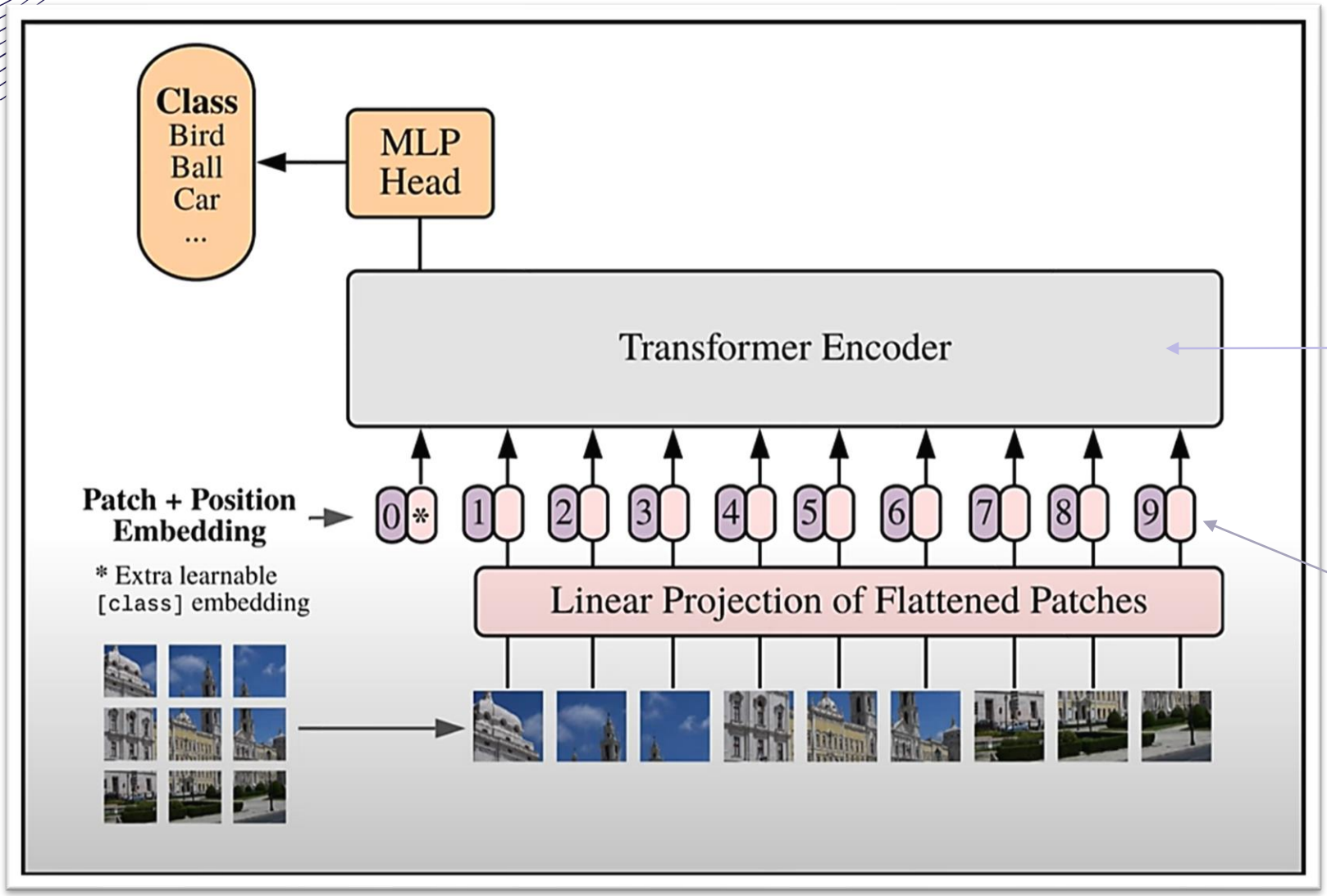
Three popular variants:

- **Encoder-Only** - useful for learning representations **BERT**
- **Decoder-Only** - useful for generation tasks **GPT-3**
- **Encoder-Decoder** - useful for sequence-to-sequence

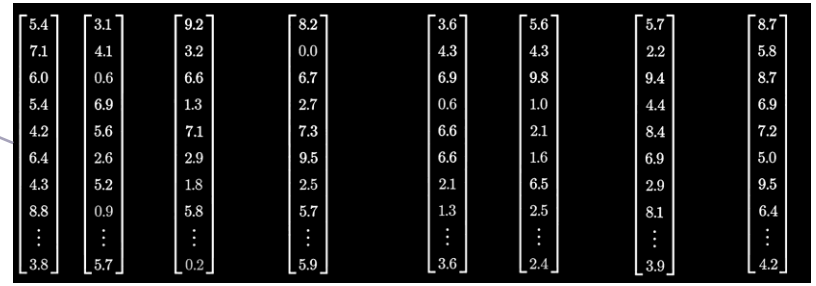




ViT: Vision Transformer (Encoder-Only)



What is going inside here?



Embedding vector representation of input patch

Mandatory videos

- 3Blue1Brown: Transformers, the tech behind LLMs | Deep Learning Chapter 5
- 3Blue1Brown: Attention in transformers, step-by-step | Deep Learning Chapter 6

- Then, exercises.

Mini-projects



Task: Soccer ball tracking

- **Purpose / learning objectives:** Get experience with data annotation, method selection, implementation, and evaluation.

- **Requirements:**
- Choose and implement both a non-NN and a NN-based method for tracking the ball in play.
- You may use the data from SoccerNet for training (go to Tracking data and labels): https://www.soccer-net.org/data
- Annotate and evaluate on a new 1-minute clip from a different environment: Either choose your own video clip from your favourite club, or take the clip we provide



Tracking data and labels

Data format

Tracklets for the players, the referees and the ball on the main camera clips

Download commands

```
mySoccerNetDownloader.downloadDataTask(task="tracking", split=["train", "test",
"challenge"])

mySoccerNetDownloader.downloadDataTask(task="tracking-2023", split=["train", "test",
"challenge"])
```

Mini-projects

Hand-in (individual):

Submit your code, your video file (in case you did not use the video from Moodle), and your annotations. Submit a pdf with text answering the following questions:

- Briefly justify why you chose the methods you did
- How did you train your models? Include description of which training data you used, and why.
- How did you annotate the test data (which tool, etc.)?
- What were the pros and cons of the NN-based vs. non-NN-based methods you implemented?
- How would you expect your model performs for ball tracking in other sports (volleyball, golf, ...)?

Evaluation:

We will use a peer-review process on Moodle, where each of you review two other submissions and give brief feedback.

Expected workload:

50 hours are assigned for the mini-project, including the evaluation.

Deadline:
November 16.

References

- **Vaswani, A., et al. (2017).** "Attention Is All You Need" *NeurIPS 2017* - Original transformer paper arXiv:1706.03762
- **Dosovitskiy, A., et al. (2020).** "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale" *ICLR 2021* - Vision Transformer (ViT), arXiv:2010.11929
- **Hochreiter, S., & Schmidhuber, J. (1997).** "Long Short-Term Memory" *Neural Computation* - LSTM architecture
- **He, K., et al. (2016).** "Deep Residual Learning for Image Recognition", *CVPR 2016* - ResNet and residual connections arXiv:1512.03385
- **Krizhevsky, A., et al. (2012).** "ImageNet Classification with Deep Convolutional Neural Networks" *NeurIPS 2012* - AlexNet
- **Simonyan, K., & Zisserman, A. (2014).** "Very Deep Convolutional Networks for Large-Scale Image Recognition", *ICLR 2015* – VGGNet, arXiv:1409.1556
- **The Illustrated Transformer** - Jay Alammar https://jalammar.github.io/illustrated-transformer/
- **Andrew Ng** - Diagrams for convolution operations